

JAVA EE 8 FRONT-END - ANGULAR (V19)

Módulo 4: Routing Moderno y Navegación

Introducción al Módulo

El enrutamiento en las arquitecturas modernas de aplicaciones de una sola página (Single Page Applications - SPAs) ha evolucionado drásticamente hacia soluciones más ligeras, eficientes y modulares. Con la consolidación del paradigma Standalone (ver ANEXO I), la configuración clásica articulada sobre `NgModule` es reemplazada por proveedores de rutas funcionales y reactivos. A lo largo de esta unidad examinarás las técnicas avanzadas de división de código (*code splitting*) por componente mediante `loadComponent` (ver ANEXO II), la captura declarativa de parámetros vía *Component Input Binding*, la securización perimetral con Functional Guards (ver ANEXO III) y la reactividad síncrona mediante Signals (ver ANEXO IV), adquiriendo las competencias requeridas para diseñar arquitecturas web escalables y optimizadas bajo las métricas de Core Web Vitals.

Estructura Temática y Hoja de Ruta Formativa

Este manual técnico está concebido como una guía teórico-práctica secuencial articulada en los siguientes puntos principales:

- **1 Configuración de rutas con componentes standalone:** Desacoplamiento del enrutador respecto a módulos, inicialización con `provideRouter`, estructura del árbol de rutas y la directiva `RouterOutlet`.
- **2 Enrutamiento con parámetros:** Definición de segmentos dinámicos, superación de la aproximación clásica (`ActivatedRoute`) e integración moderna mediante *Component Input Binding*.
- **3 Lazy loading usando loadComponent:** Estrategia de fragmentación de código (*Code Splitting*), importaciones dinámicas de ECMAScript y análisis de generación de paquetes (*chunks*).
- **4 Route Guards con funciones standalone:** Sustitución de guardias basados en clases por *Functional Guards*, inyección funcional de dependencias con `inject()` y gobierno de navegación.
- **5 Navegación programática:** Operaciones mediante el servicio `Router`, resolución de rutas relativas y transferencia de estado en memoria (*transient state*).
- **6 Ejercicio Final:** Proyecto integrador de administración de catálogo optimizado con componentes autónomos, seguridad funcional y carga diferida.
- **7 Resumen Ejecutivo del Conocimiento Adquirido:** Síntesis formal de competencias, directrices de arquitectura y patrones asimilados.

Anexos Técnicos de Referencia:

- **ANEXO I (Glosario de Términos Técnicos):** Compendio conceptual con la definición rigurosa de términos clave citados con (ver ANEXO I).

- **ANEXO II (Enrutamiento Moderno en Angular v19):** Análisis pormenorizado de rutas hijas (*children*), arquitectura de *Layouts* unificados y carga perezosa con *loadComponent*, citado con (ver ANEXO II).
- **ANEXO III (Guards en Angular v19):** Manual de Functional Guards, tipología operativa (*canActivate*, *canMatch*, etc.) y redirecciones seguras con *UrlTree*, citado con (ver ANEXO III).
- **ANEXO IV (Angular v19: El Paradigma Reactivo Basado en Signals):** Primitivas reactivas síncronas (*signal*, *computed*, *effect*), comunicación y persistencia, citado con (ver ANEXO IV).

1 Configuración de rutas con componentes standalone

El primer paso crucial en la modernización de aplicaciones web consiste en desacoplar el enrutador de los pesados módulos de arranque globales. Al eliminar el archivo central *AppModule*, la arquitectura se simplifica y se vuelve puramente orientada a componentes independientes (ver ANEXO I).

1.1 El proveedor de rutas moderno

En la arquitectura moderna de componentes standalone, el arranque de la aplicación se ejecuta de manera directa mediante la función *bootstrapApplication*. El enrutador se inicializa a través del proveedor de servicios funcional *provideRouter*. Esto reemplaza la antigua e intrusiva importación de *RouterModule.forRoot(routes)* por un modelo basado en funciones de configuración altamente escalables.

A continuación se muestra un ejemplo real de implementación del archivo de configuración global:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { appRoutes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(appRoutes)
  ]
};
```

Listing 1: src/app/app.config.ts

1.2 Estructura de declaración de rutas

El array de rutas (*appRoutes*) define la cartografía de navegación de la aplicación de manera estructurada. Cada nodo de la ruta asocia una ruta URL con su componente correspondiente o con una estrategia de carga diferida (ver ANEXO II):

```
import { Routes } from '@angular/router';
import { LayoutComponent } from '../shared/layout/layout.component';

export const appRoutes: Routes = [
  {
    path: '',
```

```
component: LayoutComponent,
children: [
  {
    path: 'inicio',
    loadComponent: () => import('./features/home/home.component')
      .then(m => m.HomeComponent)
  }
]
}
];
```

Listing 2: src/app/app.routes.ts

1.3 La directiva RouterOutlet y su integración standalone

Para renderizar las vistas dinámicas, los componentes contenedores deben importar explícitamente la directiva RouterOutlet. Al ser un desarrollo standalone, ya no dependemos de que un módulo externo declare estas dependencias; cada componente declara exactamente lo que necesita para su funcionamiento:

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-layout',
  standalone: true,
  imports: [RouterOutlet, RouterLink],
  template: `
    <header class="main-header">
      <nav>
        <a routerLink="/inicio">Inicio</a>
      </nav>
    </header>
    <main class="content-container">
      <router-outlet></router-outlet>
    </main>
  `
})
export class LayoutComponent {}
```

Listing 3: src/app/shared/layout/layout.component.ts

Miniejercicio: Configuración Completa de app.config.ts y app.routes.ts

Enunciado: Define un objeto appConfig que provea el enrutador con una ruta principal vacía que redirija a inicio y active el mapeo de parámetros por entrada de componente.

Solución:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding, Routes } from '@angular/router';

const routes: Routes = [
  { path: '', redirectTo: 'inicio', pathMatch: 'full' }
];

export const appConfig: ApplicationConfig = {
```

```
providers: [  
  provideRouter(routes, withComponentInputBinding())  
]  
};
```

Listing 4: ejercicios/p1-config/p1-app.config.ts

Explicación: La función `provideRouter` integra tanto el árbol de rutas como las características globales de enlace de componentes de forma limpia y escalable.

2 Enrutamiento con parámetros

Las aplicaciones reales requieren recuperar y representar información dinámica basándose en el estado contenido en la propia barra de direcciones (por ejemplo, el identificador único de una entidad).

2.1 Definición de parámetros dinámicos

El paso de información se realiza declarando variables con el prefijo `:` dentro del segmento de la ruta. Por ejemplo, para acceder a la ficha de un cliente, configuramos: `path: 'clientes/:id'`.

2.2 La aproximación clásica: `ActivatedRoute`

Tradicionalmente, para leer dicho parámetro, el componente hijo debía inyectar el servicio `ActivatedRoute` y suscribirse a un observable o extraer una captura instantánea (*snapshot*). Este enfoque añade dependencias directas del framework al componente y aumenta la complejidad de las pruebas unitarias:

```
@Component({ ... })  
export class DetalleClienteComponent implements OnInit {  
  clientId: string | null = null;  
  constructor(private route: ActivatedRoute) {}  
  ngOnInit() {  
    this.clientId = this.route.snapshot.paramMap.get('id');  
  }  
}
```

Listing 5: ejemplos/p2-2-detalle-cliente-clasico.component.ts

2.3 La revolución moderna: `Component Input Binding`

La aproximación moderna de Angular permite desacoplar por completo el componente de la API del enrutador. Al habilitar la característica de mapeo automático de parámetros en el arranque, el enrutador suministrará los valores de la URL directamente a las propiedades decoradas con `@Input` o declaradas como señales de entrada (`input()`, ver ANEXO I y ANEXO IV).

Para activar este comportamiento, añades el modificador `withComponentInputBinding()` en la configuración del proveedor del enrutador:

```
import { provideRouter, withComponentInputBinding } from '@angular/router';  
import { appRoutes } from './app.routes';
```

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(appRoutes, withComponentInputBinding())
  ]
};
```

Listing 6: ejemplos/p2-3-app.config.ts

Gracias a esto, el componente receptor se reduce a su mínima expresión y se vuelve altamente reutilizable:

```
import { Component, Input, OnInit } from '@angular/core';

@Component({
  selector: 'app-detalle-cliente',
  standalone: true,
  template: `<p>Visualizando datos del cliente con ID: {{ id }}</p>`
})
export class DetalleClienteComponent implements OnInit {
  @Input() id!: string; // Se asigna automáticamente desde la URL clientes/:id

  ngOnInit() {
    console.log('ID cargado de forma automática:', this.id);
  }
}
```

Listing 7: ejemplos/p2-3-detalle-cliente.component.ts

Ejercicio Parcial: Captura Reactiva de Parámetros Múltiples de Ruta

Enunciado

Configura una vista de filtrado avanzado de productos que requiera dos parámetros dinámicos simultáneos en la URL: el código de la categoría y el identificador de la marca (ej. productos/:categoria/:marca). Resuelve la captura de estos valores de manera puramente standalone utilizando *Component Input Binding*.

Solución

En primer lugar, define la ruta estructurada con dos segmentos variables:

```
{
  path: 'productos/:categoria/:marca',
  loadComponent: () => import('./filtrado.component').then(m => m.FiltradoComponent)
}
```

Listing 8: ejercicios/p2-parcial/p2-filtrado.routes.ts

Posteriormente, en el componente standalone, captura los parámetros como entradas directas mapeadas de forma implícita por el framework:

```
import { Component, Input, OnInit } from '@angular/core';

@Component({
  selector: 'app-filtrado',
```

```
standalone: true,
template: `
  <div class="filter-results">
    <h2>Resultados de Búsqueda</h2>
    <p>Categoria activa: <strong>{{ categoria }}</strong></p>
    <p>Marca seleccionada: <strong>{{ marca }}</strong></p>
  </div>
`
})
export class FiltradoComponent implements OnInit {
  @Input() categoria!: string; // Mapeado de :categoria
  @Input() marca!: string;     // Mapeado de :marca

  ngOnInit() {
    console.log(`Buscando ${this.categoria} filtrado por la marca ${this.marca}`);
  }
}
```

Listing 9: ejercicios/p2-parcial/filtrado.component.ts

Explicación

- **Mapeo Directo y Transparente:** El modificador `withComponentInputBinding()` mapea los segmentos nominales de la ruta a las propiedades `@Input()` que posean el mismo nombre, sin requerir suscripciones a observables.
- **Aislamiento y Reusabilidad:** El componente queda completamente desacoplado del servicio `ActivatedRoute`, lo que permite probarlo y reutilizarlo como un componente autónomo estándar.

3 Lazy loading usando `loadComponent`

El rendimiento inicial de la aplicación depende en gran medida de su tamaño de descarga inicial. Si cargas todos los componentes en el paquete raíz, experimentarás tiempos de carga lentos.

3.1 Estrategia de división del código (Code Splitting)

La carga perezosa (*Lazy Loading*, ver ANEXO I) permite posponer la descarga, el parseado y la compilación de un componente en el navegador web hasta el momento exacto en que intentas acceder a la ruta asociada. El navegador descarga únicamente el bloque mínimo indispensable de la pantalla inicial.

3.2 Implementación con importación dinámica

Con las arquitecturas tradicionales orientadas a módulos, la carga diferida requería declarar un enrutador secundario completo (`loadChildren`). En el ecosistema moderno standalone, se implementa directamente por componente mediante la propiedad `loadComponent` combinada con la directiva nativa de importación dinámica de ECMAScript:

```
export const routes: Routes = [
  {
    path: 'panel-control',
```

```
loadComponent: () => import('./features/admin/panel.component')
  .then(m => m.PanelComponent)
}
];
```

Listing 10: ejemplos/p3-2-lazy.routes.ts

3.3 Análisis de generación de Chunks

Durante el proceso de compilación, herramientas modernas analizan estas sentencias de importación dinámica. En lugar de compilar un único archivo consolidado, generan *chunks* individuales de código JavaScript independientes (ej. `panel.component-XYZ123.js`). El navegador realiza una petición de red para adquirir este fragmento exclusivamente al activar la transición a dicha sección.

Miniejercicio: Ruta Perezosa Avanzada con Tipado y Prefetch

Enunciado: Define una ruta perezosa para el componente `ReportesComponent` utilizando exportación por defecto y aplicando precarga inteligente de chunks.

Solución:

```
{
  path: 'reportes',
  loadComponent: () => import('./features/reportes/reportes.component')
}
```

Listing 11: ejercicios/p3-reportes/reportes.routes.ts

Explicación: La combinación de importación dinámica y componentes standalone permite aislar bloques pesados de gráficos sin sobrecargar el bundle principal.

4 Route Guards con funciones standalone

Los sistemas de seguridad en el lado del cliente garantizan que los usuarios no naveguen a pantallas para las cuales no poseen privilegios de acceso (por ejemplo, pasarelas de pago o paneles de administración).

4.1 La transición a los Functional Guards

Anteriormente, los mecanismos de seguridad requerían crear clases que implementaran complejas interfaces (`CanActivate`). En las versiones actuales del framework, este enfoque estructurado ha sido reemplazado por los **Functional Guards** (ver ANEXO III). Los guardianes funcionales son simplemente funciones de TypeScript puras que devuelven un booleano, un árbol de navegación alternativo (`UrlTree`), una promesa o un observable.

4.2 La inyección funcional de dependencias

Dado que un guardián funcional no es una clase, carece de un método constructor tradicional para resolver dependencias. Para solventar esto, utilizas el sistema de inyección reactiva mediante la función global `inject()`.

A continuación se muestra una implementación empresarial de un guardián de autorización funcional:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isSessionActive()) {
    return true; // Acceso concedido
  }

  // Redirección segura a la pantalla de Login retornando un UrlTree
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url }
  });
};
```

Listing 12: src/app/core/guards/auth.guard.ts

Ejercicio Parcial: Control de Acceso por Roles mediante Guards Funcionales

Enunciado

Desarrolla un sistema de seguridad de tipo guardián funcional standalone llamado `roleGuard`. Este guardián debe evaluar si el usuario tiene sesión activa y si posee el rol administrativo requerido ('ADMIN'). En caso negativo, debe impedir el acceso y redirigir al usuario hacia un componente de «Acceso Denegado».

Solución

Implementa la solución utilizando el enfoque funcional nativo y el inyector dinámico de dependencias:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from './auth.service';

export const roleGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  // Comprobar la existencia del usuario y su rol administrativo
  if (authService.isAuthenticated() && authService.getUserRole() === 'ADMIN') {
    return true; // Permitir el transito a la ruta
  }

  // Redirigir de manera segura si no cumple las condiciones
  return router.createUrlTree(['/acceso-denegado']);
};
```

Listing 13: ejercicios/p4-roles/role.guard.ts

Explicación

- **Inyección Funcional:** Utilizas la función `inject()` para acceder tanto al servicio de autenticación (`AuthService`) como al servicio de enrutamiento (`Router`) directamente dentro del cuerpo de la función.
- **Redirección Atómica con `UrlTree`:** Retornar un objeto `UrlTree` cancela la navegación actual y fuerza una redirección limpia hacia la vista alternativa de acceso denegado.

5 Navegación programática

Existen situaciones operativas en las que debes redirigir al usuario no a través de una acción física directa sobre un enlace, sino como resultado directo de una lógica de negocio resuelta.

5.1 Uso avanzado del servicio Router

Para manipular el historial de navegación por código, inyectas el servicio global de enrutador mediante `inject(Router)`. Este servicio dispone de métodos clave para desencadenar tránsitos de URL:

- `navigate()`: Permite especificar la dirección de destino a través de un array de segmentos jerárquicos de ruta (ej. `this.router.navigate(['/clientes', id, 'detalle'])`).
- `navigateByUrl()`: Recibe un string absoluto con el recorrido directo.

5.2 Contextualización de rutas relativas

Durante la navegación programática dentro de componentes profundamente anidados, es indispensable mantener la referencia de origen. Para evitar escribir rutas absolutas complejas, configuras la propiedad `relativeTo` recuperando la instancia del servicio `ActivatedRoute`:

```
import { Component, inject } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';

@Component({ ... })
export class EditorComponent {
  private router = inject(Router);
  private activeRoute = inject(ActivatedRoute);

  cancelarEdicion() {
    // Retorna una seccion atras relativo a la ruta activa actual
    this.router.navigate(['../'], { relativeTo: this.activeRoute });
  }
}
```

Listing 14: `src/app/features/editor/editor.component.ts`

5.3 Paso y persistencia de estado transitorio

La navegación programática moderna permite adjuntar información contextual transitoria a través de la propiedad de configuración `state`. Esta persistencia se produce de manera exclusiva en memoria durante el ciclo de vida del tránsito, evitando exponer información sensible en los parámetros públicos de la URL:

```
// Desde el componente emisor
this.router.navigate(['/resumen'], {
  state: { transaccionExitosa: true, hashOperativo: '0x992B' }
});

// Desde el componente receptor
const navegacion = this.router.getCurrentNavigation();
const estado = navegacion?.extras.state as { transaccionExitosa: boolean; hashOperativo: string };
```

Listing 15: ejemplos/p5-3-navegacion-estado.ts

Miniejercicio: Redirección con Parámetros de Consulta (Query Params)

Enunciado: Implementa un método en un componente que redirija por código hacia /productos pasando el filtro orden=desc y limite=20 en la URL.

Solución:

```
aplicarFiltros() {
  this.router.navigate(['/productos'], {
    queryParams: { orden: 'desc', limite: 20 }
  });
}
```

Listing 16: ejercicios/p5-query/productos-query.component.ts

Explicación: La propiedad queryParams serializa automáticamente los pares clave-valor en la query string de la dirección web.

6 Ejercicio Final

Sistema de Gestión y Catálogo de Productos Profesionales

Enunciado

Construye un sistema integral para la administración de un catálogo de productos profesionales articulado a través de un árbol de vistas standalone de última generación que cumpla los siguientes requisitos:

1. **Servicio de Autenticación Reactivo:** Diseña un servicio singleton (AuthService) que gestione el estado de sesión mediante Signals (signal<boolean>).
2. **Guardián Funcional de Acceso:** Implementa un guardián funcional (authGuard) que proteja la ruta de creación de productos y redirija al login si la sesión no está activa.
3. **Cartografía de Rutas con División de Código:** Configura app.routes.ts empleando loadComponent para habilitar la fragmentación perezosa (*lazy chunks*) bajo demanda.
4. **Vista de Listado (ProductListComponent):** Renderiza la matriz dinámica de productos con el nuevo flujo declarativo @for y enlaces para consultar la ficha indivi-

dual.

5. **Vista de Detalle (ProductDetailComponent):** Captura el identificador dinámico (:id) exclusivamente mediante *Component Input Binding*.
6. **Vista de Creación y Registro (ProductCreateComponent):** Formulario interactivo que, tras la confirmación, ejecute una navegación programática retornando al catálogo e inyectando un estado temporal (state) en memoria.

Solución

Paso 1: El Servicio de Autenticación Reactivo

```
import { Injectable, signal } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private isLoggedIn = signal<boolean>(false);

  isSessionActive(): boolean {
    return this.isLoggedIn();
  }

  login(): void {
    this.isLoggedIn.set(true);
  }

  logout(): void {
    this.isLoggedIn.set(false);
  }
}
```

Listing 17: src/app/core/services/auth.service.ts

Paso 2: El Guardián Funcional de Acceso

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isSessionActive()) {
    return true;
  }

  return router.createUrlTree(['/login']);
};
```

Listing 18: src/app/core/guards/catalogo-auth.guard.ts

Paso 3: Fichero de Enrutamiento Central

```
import { Routes } from '@angular/router';
```

```
import { authGuard } from '../../../core/guards/catalogo-auth.guard';

export const appRoutes: Routes = [
  {
    path: '',
    redirectTo: 'productos',
    pathMatch: 'full'
  },
  {
    path: 'productos',
    loadComponent: () => import('./product-list.component')
      .then(m => m.ProductListComponent)
  },
  {
    path: 'productos/nuevo',
    loadComponent: () => import('./product-create.component')
      .then(m => m.ProductCreateComponent),
    canActivate: [authGuard]
  },
  {
    path: 'productos/:id',
    loadComponent: () => import('./product-detail.component')
      .then(m => m.ProductDetailComponent)
  },
  {
    path: 'login',
    loadComponent: () => import('./login.component')
      .then(m => m.LoginComponent)
  }
];
```

Listing 19: src/app/features/catalogo/catalogo.routes.ts

Paso 4: Vista de Listado de Productos

```
import { Component, inject } from '@angular/core';
import { Router, RouterLink } from '@angular/router';

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [RouterLink],
  template: `
    <div class="container">
      <h2>Catalogo de Productos Profesionales</h2>
      <button class="btn btn-primary" routerLink="/productos/nuevo">
        Agregar Nuevo Producto
      </button>
      <ul class="product-grid">
        @for (product of products; track product.id) {
          <li class="product-card">
            <h4>{{ product.nombre }}</h4>
            <p>Precio: {{ product.precio }} EUR</p>
            <button class="btn-detail" (click)="verDetalle(product.id)">
              Ver Detalles Ficha
            </button>
          </li>
        }
      </ul>
    </div>
```

```

        }
      </ul>
    </div>
  },
  ...
})
export class ProductListComponent {
  private router = inject(Router);

  products = [
    { id: 'p01', nombre: 'Servidor Computacion Cloud', precio: 1200 },
    { id: 'p02', nombre: 'Switch de Red Gestionable', precio: 340 },
    { id: 'p03', nombre: 'Terminal de Operario IoT', precio: 150 }
  ];

  verDetalle(id: string): void {
    this.router.navigate(['/productos', id]);
  }
}

```

Listing 20: src/app/features/catalogo/product-list.component.ts

Paso 5: Vista de Detalle con Component Input Binding

```

import { Component, Input, OnInit } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-product-detail',
  standalone: true,
  imports: [RouterLink],
  template: `
    <div class="container detail-container">
      <h2>Ficha del Producto</h2>
      <p>Buscando especificaciones del codigo de producto: <strong>{{ id }}</strong></p>
      <div class="mock-data">
        <p>Estado de inventario: <span style="color: green;">Disponible</span></p>
        <p>Certificaciones tecnicas: CE, FCC, RoHS</p>
      </div>
      <a routerLink="/productos" class="btn-back">Regresar al catalogo</a>
    </div>
  `
})
export class ProductDetailComponent implements OnInit {
  @Input() id!: string;

  ngOnInit() {
    console.log(`Cargando informacion del producto: ${this.id}`);
  }
}

```

Listing 21: src/app/features/catalogo/product-detail.component.ts

Paso 6: Vista de Creación con Redirección Programática

```

import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';

@Component({

```

```

selector: 'app-product-create',
standalone: true,
template: `
  <div class="container form-container">
    <h2>Registrar Nuevo Producto en Catalogo</h2>
    <form (submit)="guardarProducto($event)">
      <div class="form-group">
        <label for="name">Nombre del Producto: </label>
        <input type="text" id="name" required placeholder="Ej. Hub RJ45">
      </div>
      <div class="form-group">
        <label for="price">Precio (EUR): </label>
        <input type="number" id="price" required placeholder="Ej. 199">
      </div>
      <button type="submit" class="btn btn-save">Confirmar Guardado</button>
    </form>
  </div>
`
})
export class ProductCreateComponent {
  private router = inject(Router);

  guardarProducto(event: Event): void {
    event.preventDefault();
    console.log('Persistiendo producto en base de datos...');
    this.router.navigate(['/productos'], {
      state: { mensaje: 'Producto registrado exitosamente en el catalogo.' }
    });
  }
}

```

Listing 22: src/app/features/catalogo/product-create.component.ts

Explicación

- **Arquitectura Standalone y Lazy Loading:** Cada vista funcional se descarga bajo demanda mediante `loadComponent`, optimizando el paquete inicial y cumpliendo con Core Web Vitals.
- **Seguridad Perimetral Funcional:** El guardián `authGuard` protege la ruta mediante funciones puras resolviendo dependencias con `inject()`.
- **Mapeo Moderno y Estado en Memoria:** Se utiliza *Component Input Binding* para capturar los parámetros de URL de forma limpia y transparente.

7 Resumen Ejecutivo del Conocimiento Adquirido

La consolidación del enrutamiento moderno en Angular 19 establece un marco de trabajo de máxima eficiencia y escalabilidad técnica para el desarrollo frontend:

- **Arquitectura Libre de Módulos:** La adopción de componentes standalone y la inicialización funcional con `provideRouter` eliminan el código redundante de los antiguos `NgModule`.
- **Optimización Granular con Lazy Loading:** La fragmentación de paquetes a nivel de componente mediante `loadComponent` asegura que los recursos se transfieran exclusivamente

al solicitar la vista activa.

- **Desacoplamiento con Component Input Binding:** El mapeo directo de parámetros de ruta a propiedades `@Input()` simplifica la estructura de los componentes y agiliza las pruebas.
- **Seguridad Funcional y Reactiva:** Los Functional Guards permiten proteger rutas mediante funciones puras, resolviendo dependencias en línea mediante `inject()`.

ANEXO I: Glosario de Términos Técnicos

1. **Single Page Application (SPA):** Aplicación web donde el navegador carga una única página base y delega la actualización visual dinámica y la navegación en memoria al framework JavaScript.
2. **Standalone Component:** Componente autónomo en Angular que declara directamente sus dependencias en sus propios metadatos sin intermediación de un archivo `NgModule`.
3. **provideRouter:** Función proveedora utilizada en el arranque para configurar e inicializar el sistema de enrutamiento global en entornos desacoplados de módulos.
4. **Lazy Loading (Carga Perezosa):** Estrategia de optimización que posterga la descarga, compilación y renderizado de un componente hasta el momento exacto en que intentas acceder a su ruta asociada.
5. **loadComponent:** Propiedad del enrutador de Angular que define la carga diferida directa de un componente autónomo mediante una importación dinámica asíncrona de ECMAScript.
6. **RouterOutlet:** Directiva del framework (`<router-outlet>`) que actúa como marcador de posición en la plantilla visual para la inyección bajo demanda del componente activado.
7. **RouterLink:** Directiva aplicada a hipervínculos que intercepta el evento de clic para gestionar la navegación en memoria sin recargar la página web completa.
8. **Component Input Binding:** Característica moderna de Angular que asigna automáticamente parámetros de ruta y de consulta a las entradas (`@Input` o `input()`) del componente.
9. **Functional Guard:** Función pura de TypeScript utilizada para gobernar el acceso, resolución o salida de rutas, reemplazando a los guardias clásicos basados en clases.
10. **UrlTree:** Estructura de datos interna e inmutable de Angular que representa y manipula URLs de forma estructurada; al ser retornada por un guardián funcional, ejecuta una redirección automática.
11. **Signals:** Primitivas reactivas síncronas de granularidad fina que notifican de forma directa a sus consumidores sobre alteraciones de estado con coste constante $\mathcal{O}(k)$.
12. **Code Splitting (División de Código):** Mecanismo de optimización en compilación que fragmenta el paquete de la aplicación en múltiples archivos (*chunks*) independientes transferibles bajo demanda.
13. **Injection Context:** Contexto específico de ejecución donde el inyector activo de Angular está disponible para invocar la función `inject()`.
14. **Tree-Shaking:** Proceso automático de optimización que analiza las sentencias de importación y elimina del paquete final el código muerto o no consumido.

ANEXO II: Enrutamiento Moderno en Angular v19

Conceptos Esenciales, Rutas Hijas y Lazy Loading

En el desarrollo moderno de aplicaciones web de una sola página (SPA), la experiencia de usuario se caracteriza por una fluidez absoluta. A diferencia de los sitios web tradicionales, el navegador web nunca recarga completamente el documento HTML al cambiar de sección o menú.

1 La Analogía del Ascensor

Imagina que tu aplicación es un edificio corporativo:

- **La URL:** Es el botón físico del ascensor que pulsas para dirigirte a un lugar concreto.
- **El Enrutador (Router):** Es el cerebro inteligente del ascensor que decide a qué piso trasladarte.
- **Los Componentes:** Son las oficinas ubicadas en los distintos pisos que se revelan al abrirse las puertas.

Miniejercicio: Asociación Conceptual de Ruta Directa

Enunciado: Define una ruta directa en `app.routes.ts` que mapee la URL `soporte` al componente standalone `SoporteComponent`.

Solución:

```
{
  path: 'soporte',
  component: SoporteComponent
}
```

Listing 23: anexos/a2-1-soporte.routes.ts

Explicación: En la analogía del ascensor, pulsar el botón `soporte` abre directamente las puertas de la dependencia `SoporteComponent` sin intermediación de módulos.

2 El Concepto de Rutas Hijas (children)

Para resolver patrones de diseño con plantillas comunes (*Layout*), Angular introduce la propiedad `children` dentro del objeto de configuración de rutas, permitiendo anidar rutas jerárquicamente:

```
import { Routes } from '@angular/router';
import { LayoutComponent } from '../core/layout/layout.component';

export const appRoutes: Routes = [
  {
    path: '',
    component: LayoutComponent,
    children: [
      {
        path: 'inicio',
        loadChildren: () => import('../features/home/home.component')
      }
    ]
  }
]
```

```

        .then(m => m.HomeComponent)
    }
  ]
}
];

```

Listing 24: anexos/a2-2-layout.routes.ts

Miniejercicio: Estructuración de Rutas Hijas Múltiples

Enunciado: Configura una rama padre cuenta que contenga dos rutas hijas: perfil y seguridad.

Solución:

```

{
  path: 'cuenta',
  component: CuentaLayoutComponent,
  children: [
    { path: 'perfil', component: PerfilComponent },
    { path: 'seguridad', component: SeguridadComponent }
  ]
}

```

Listing 25: anexos/a2-2-cuenta-hijas.routes.ts

Explicación: Ambas vistas comparten el diseño de CuentaLayoutComponent, cargándose dentro de su respectivo <router-outlet>.

3 Análisis Detallado de la Configuración y Chunks

La propiedad `loadComponent` permite dividir el código en múltiples archivos independientes (*chunks*), descargando componentes únicamente cuando el usuario navega a ellos.

Miniejercicio: Ruta Perezosa con Exportación por Defecto

Enunciado: Declara una ruta perezosa para el componente PerfilComponent asumiendo exportación por defecto (`export default`).

Solución:

```

{
  path: 'perfil',
  loadComponent: () => import('./features/perfil/perfil.component')
}

```

Listing 26: anexos/a2-3-perfil-default.routes.ts

Explicación: Si el componente se exporta con `export default`, puedes omitir la llamada `.then()`, simplificando la sintaxis del chunk.

4 Implementación y Anatomía del LayoutComponent

Los componentes de diseño combinan plantillas inline y directivas de enrutamiento:

```

import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

```

```
@Component({
  selector: 'app-layout',
  standalone: true,
  imports: [RouterOutlet, RouterLink],
  template: `
    <header class="app-header">
      <nav class="navigation-bar">
        <a routerLink="/inicio" class="nav-link">Inicio</a>
      </nav>
    </header>
    <main class="content-area">
      <router-outlet></router-outlet>
    </main>
  `
})
export class LayoutComponent {}
```

Listing 27: anexos/a2-4-layout.component.ts

Miniejercicio: Enlace Activo con RouterLinkActive

Enunciado: Importa y aplica la directiva RouterLinkActive para resaltar el enlace activo con la clase CSS active-link.

Solución:

```
@Component({
  imports: [RouterLink, RouterLinkActive],
  template: `<a routerLink="/inicio" routerLinkActive="active-link">Inicio</a>`
})
export class NavComponent {}
```

Listing 28: anexos/a2-4-link-activo.component.ts

Explicación: RouterLinkActive inspecciona la URL activa y añade la clase visual de forma reactiva al coincidir con el destino.

5 El Rol Esencial de <router-outlet>

La etiqueta <router-outlet></router-outlet> actúa como marcador dinámico en el DOM donde se inyectan los componentes hijos correspondientes a la ruta activa.

Miniejercicio: Contenedor Dinámico Múltiple

Enunciado: Configura un componente con múltiples salidas de enrutamiento nombradas secundarias si fuera preciso.

Solución:

```
<router-outlet name="auxiliar"></router-outlet>
```

Listing 29: anexos/a2-5-outlet-secundario.component.html

Explicación: Permite proyectar vistas secundarias de manera simultánea en la misma pantalla.

6 Ejercicio Final Enrutamiento

Portal de Administración con Layout Maestro y Rutas Hijas Perezosas

Enunciado

Diseña la arquitectura de enrutamiento completa para un portal administrativo corporativo. Debes crear un contenedor maestro (AdminLayoutComponent) y configurar rutas hijas perezosas para métricas y usuarios.

Solución

```
import { Routes } from '@angular/router';
import { AdminLayoutComponent } from './admin-layout.component';

export const adminRoutes: Routes = [
  {
    path: 'admin',
    component: AdminLayoutComponent,
    children: [
      { path: '', redirectTo: 'metricas', pathMatch: 'full' },
      {
        path: 'metricas',
        loadChildren: () => import('./features/metricas.component')
          .then(m => m.MetricasComponent)
      }
    ]
  }
];
```

Listing 30: anexos/a2-6-admin.routes.ts

Explicación

- **Estructura Limpia:** El layout maestro permanece montado mientras alternas vistas hijas.
- **Lazy Chunks:** Cada submódulo se aísla en su propio archivo JavaScript bajo demanda.

ANEXO III: Guards en Angular v19

Manual Docente y Configuración del Cambio de Paradigma

En Angular v19, los guardias basados en clases están obsoletos y el estándar absoluto son los **Functional Guards**, funciones de TypeScript puras y ligeras.

1 Tipología Operativa de Guards

1. **canActivate**: Controla si puedes acceder a una ruta específica.
2. **canActivateChild**: Protege en bloque todas las rutas hijas de una ruta principal.
3. **canDeactivate**: Controla si puedes abandonar la ruta actual (ideal para formularios).
4. **canMatch**: Decide si una ruta coincide con la URL antes de evaluarla.

2 Ejemplo Práctico de Funcional Guard

Implementación moderna mediante `inject()` y retorno de `UrlTree`:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isLoggedIn()) {
    return true;
  }
  return router.parseUrl('/login');
};
```

Listing 31: anexos/a3-2-auth.guard.ts

ANEXO IV: Angular v19: El Paradigma Reactivo Basado en Signals

Reactividad Síncrona, Primitivas y Carrito Integrado

Angular v19 consolida la adopción de los Signals como pilar reactivo fundamental, permitiendo actualizaciones de granularidad fina $\mathcal{O}(k)$ sin la sobrecarga de Zone.js.

1 Diferencias Esenciales: Signals frente a RxJS

- **Signals:** Diseñados para reactividad de estado síncrona y propagación inmediata a plantillas.
- **RxJS:** Reservado para flujos asíncronos complejos, web sockets y peticiones HTTP.

2 Anatomía de los Signals en Angular v19

2.1 Signals de Escritura (Writable Signals)

Se inicializan con `signal(valorInicial)` y se modifican mediante `.set()` o `.update()`.

2.2 Signals Derivados (Computed Signals)

Permiten calcular valores derivados de solo lectura con memoización y evaluación perezosa (`computed()`).

2.3 Efectos Colaterales (Effects)

Ejecutan código automáticamente ante cambios de estado reactivo (`effect()`).

3 Emisión de Eventos con @Output()

Mecanismo tradicional plenamente compatible para la comunicación ascendente mediante `EventEmitter`.

4 Inyección Funcional y Declaración en Campo de Clase

En Angular v19, los efectos y servicios se declaran directamente como propiedades de clase utilizando `inject()`.

5 Ejercicio Final Signals

Carrito de Compras de Alta Fidelidad con Signals

Enunciado

Diseña un componente de carrito de compras reactivo y autosuficiente que gestione una colección interactiva en memoria, calcule subtotales e impuestos en tiempo real mediante `computed()`, sincronice el estado en `localStorage` mediante `effect()` y utilice el nuevo flujo de control declarativo de Angular.

Solución

```
import { Component, signal, computed, effect } from '@angular/core';

@Component({
  selector: 'app-carrito',
  standalone: true,
  template: `<div>Total Compra: {{ total() }} EUR</div>`
})
export class CarritoComprasComponent {
  items = signal<{ precio: number; cantidad: number }[]>([
    { precio: 40, cantidad: 2 }
  ]);

  subtotal = computed(() => this.items().reduce((acc, i) => acc + (i.precio * i.cantidad), 0));
  total = computed(() => this.subtotal() * 1.21);

  private persistir = effect(() => {
    localStorage.setItem('cart', JSON.stringify(this.items()));
  });
}
```

Listing 32: anexos/a4-5-carrito-compras.component.ts

Explicación

- **Precisión Granular:** La actualización computada de subtotales con `computed()` evita barridos masivos en el árbol de componentes.
- **Efectos Seguros:** La persistencia automática en `localStorage` se efectúa mediante `effect()` ante mutaciones reales de estado.